

Parametric Curves & Bézier Curves

From Lines to Smooth Curves in Computer Graphics

Ashrafur Rahman

Adjunct Lecturer

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology (BUET)

Introduction

Parametric Lines

Higher Order Polynomials

Curve Continuity

Bézier Curves

General Bézier Curves

Bernstein Polynomials

Matrix Representation

Introduction

Raster vs Vector

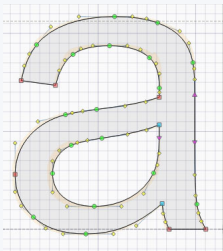
By now you know that images are composed of pixels. That is why if you zoom in on an image, it will look pixelated. This is because the image is made of pixels, and when you zoom in, you are just enlarging the pixels.

But some stuff like fonts and logos are not made of pixels, but rather of curves. They can be zoomed in infinitely without losing quality.

Raster vs Vector

By now you know that images are composed of pixels. That is why if you zoom in on an image, it will look pixelated. This is because the image is made of pixels, and when you zoom in, you are just enlarging the pixels.

But some stuff like fonts and logos are not made of pixels, but rather of curves. They can be zoomed in infinitely without losing quality.

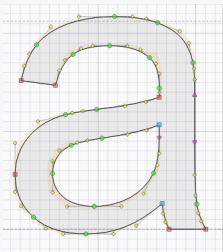


(a) Representation of a Font

Raster vs Vector

By now you know that images are composed of pixels. That is why if you zoom in on an image, it will look pixelated. This is because the image is made of pixels, and when you zoom in, you are just enlarging the pixels.

But some stuff like fonts and logos are not made of pixels, but rather of curves. They can be zoomed in infinitely without losing quality.



(a) Representation of a Font



Raster
GIF, JPEG, PNG



Vector
SVG

(b) A bitmap logo vs an SVG logo

Why Parametric Curves?

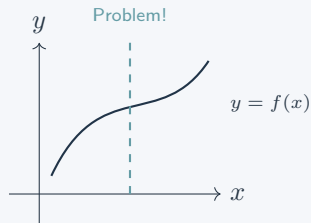
Hard to Draw

Explicit form: $y = f(x)$

- Single-valued functions only
- Cannot represent vertical lines
- Limited to function graphs

Implicit form: $F(x, y) = 0$

- Difficult to sample points
- No direction information



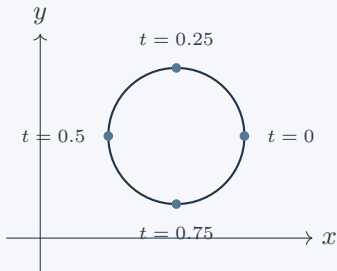
Cannot represent vertical lines

Parametric Representation: The Solution

Parametric Form

$$\mathbf{P}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$$

Where $t \in [0, 1]$ is the parameter



Advantages of Parametric Curves

- Can represent any curve shape (circles, loops, vertical lines)
- Natural parameterization for animations, [desmos demo](#).
- Easy to sample points uniformly
- Provides direction and velocity information
- Intuitive control through control points (will see later)

Applications in Computer Graphics

2D Graphics

- **Font Rendering:** TrueType, OpenType fonts
- **Vector Graphics:** SVG, Adobe Illustrator
- **UI Design:** Smooth transitions, icons
- **Logo Design:** Scalable curves

Animation & Motion

- **Motion Paths:** Object trajectories
- **Camera Movement:** Smooth camera paths
- **Interpolation:** Between keyframes
- **Easing Functions:** CSS animations

3D Modeling

- **Surface Generation:** NURBS surfaces
- **CAD Systems:** Engineering design
- **Game Development:** Character modeling
- **Architecture:** Curved structures

Parametric Lines

Parametric Line Representation

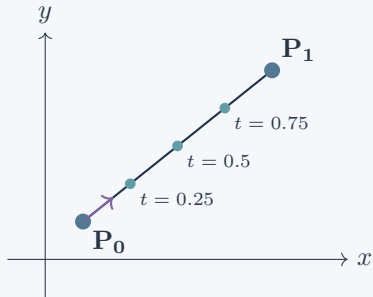
Linear Parametric Equation

Given two points $\mathbf{P}_0$ and $\mathbf{P}_1$:

$$\begin{aligned}\mathbf{P}(t) &= \mathbf{P}_0 + t(\mathbf{P}_1 - \mathbf{P}_0) \\ &= (1 - t)\mathbf{P}_0 + t\mathbf{P}_1\end{aligned}$$

Where $t \in [0, 1]$:

- $t = 0 \Rightarrow \mathbf{P}(0) = \mathbf{P}_0$
- $t = 1 \Rightarrow \mathbf{P}(1) = \mathbf{P}_1$
- $t = 0.5 \Rightarrow$ midpoint



Linear Interpolation (LERP)

LERP Function

Linear interpolation between two values:

$$\text{lerp}(A, B, t) = (1 - t)A + tB$$

Properties:

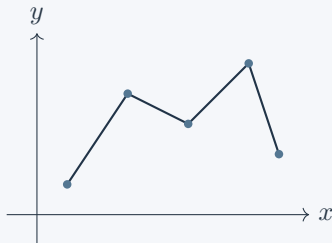
- $\text{lerp}(A, B, 0) = A$
- $\text{lerp}(A, B, 1) = B$
- Linear relationship

Code Example

```
def lerp(A, B, t):  
    return (1 - t) * A + t * B
```

Interactive Demo

[Desmos Demo: Piecewise Lines]



Piecewise Linear Curve

Higher Order Polynomials

From Linear to Polynomial

Polynomial Curves

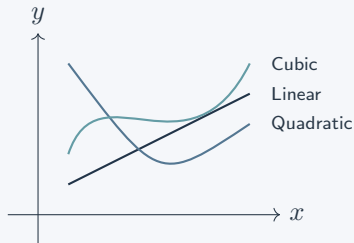
Linear: $P(t) = a_0 + a_1t$

Quadratic: $P(t) = a_0 + a_1t + a_2t^2$

Cubic: $P(t) = a_0 + a_1t + a_2t^2 + a_3t^3$

Benefits:

- More control over shape
- Smoother curves
- Higher flexibility



[Desmos Demo: Piecewise Polynomials]

The Control Problem

Challenges with Polynomials

Unintuitive Control:

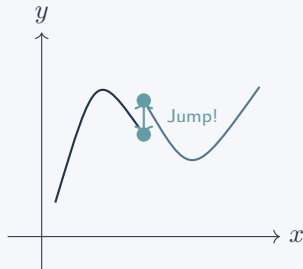
- Coefficients a_0, a_1, a_2, a_3 don't correspond to visual features
- Hard to predict curve shape from coefficients
- Difficult to achieve desired curves

Global Effects:

- Changing one coefficient affects entire curve
- No local control
- Unpredictable modifications

Piecewise Issues:

- Discontinuities at joints
- Visible "kinks" or jumps
- Not visually pleasing



Discontinuous Connection

Curve Continuity

C^0 Continuity: Position only

C^0 Continuity

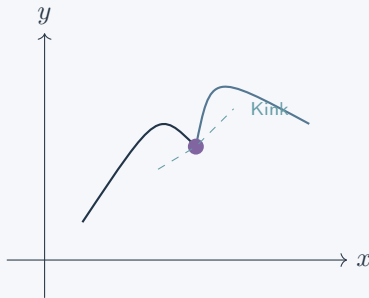
The curves meet at the same point:

$$\mathbf{P}_1(1) = \mathbf{P}_2(0)$$

but tangents may differ.

Properties:

- No gaps in the curve
- Might have sharp change in direction (kink)
- Minimum requirement for connectedness



C^1 Continuity: Tangent but curvature jump

C^1 Continuity

Same position and same tangent:

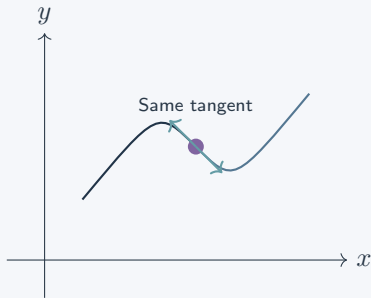
$$\mathbf{P}_1(1) = \mathbf{P}_2(0),$$

$$\mathbf{P}_1'(1) = \mathbf{P}_2'(0)$$

but different curvature.

Properties:

- No visible kink
- Smooth tangent transition



C^2 Continuity: Tangent and curvature match

C^2 Continuity

Same position, tangent, and curvature:

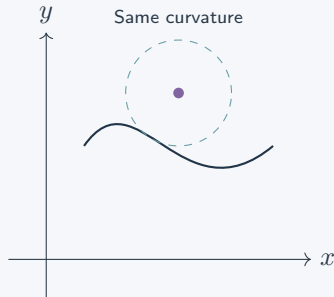
$$\mathbf{P}_1(1) = \mathbf{P}_2(0)$$

$$\mathbf{P}_1'(1) = \mathbf{P}_2'(0)$$

$$\mathbf{P}_1''(1) = \mathbf{P}_2''(0)$$

Properties:

- Perfectly smooth
- Highest visual quality



Bézier Curves

Quadratic Bézier: LERP of LERPs

Construction Method

Given three control points P_0 , P_1 , P_2 :

Step 1: Linear interpolation

$$Q_0(t) = \text{lerp}(P_0, P_1, t)$$

$$Q_1(t) = \text{lerp}(P_1, P_2, t)$$

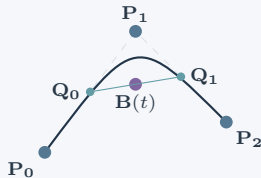
Step 2: Another linear interpolation

$$B(t) = \text{lerp}(Q_0(t), Q_1(t), t)$$

$$= (1-t)^2 P_0 + 2t(1-t) P_1 + t^2 P_2$$

Definition:

$$\text{quadBezier}(A, B, C, t) = (1-t)^2 A + 2t(1-t) B + t^2 C$$



Quadratic Bézier: Mathematical Form

Expanded Form

Starting from the LERP of LERPs:

$$\begin{aligned}\mathbf{B}(t) &= (1-t)[(1-t)\mathbf{P}_0 + t\mathbf{P}_1] + t[(1-t)\mathbf{P}_1 + t\mathbf{P}_2] \\ &= (1-t)^2\mathbf{P}_0 + 2t(1-t)\mathbf{P}_1 + t^2\mathbf{P}_2\end{aligned}$$

Bernstein Form:

$$\mathbf{B}(t) = B_{0,2}(t)\mathbf{P}_0 + B_{1,2}(t)\mathbf{P}_1 + B_{2,2}(t)\mathbf{P}_2$$

Where the Bernstein polynomials are:

$$B_{0,2}(t) = (1-t)^2 \quad B_{1,2}(t) = 2t(1-t) \quad B_{2,2}(t) = t^2$$

Key Properties

- **Endpoint Interpolation:** $\mathbf{B}(0) = \mathbf{P}_0$, $\mathbf{B}(1) = \mathbf{P}_2$
- **Tangent Control:** $\mathbf{P}_1$ controls the tangent direction

Cubic Bézier: LERP of Quadratics

Construction Method

Four control points P_0, P_1, P_2, P_3 :

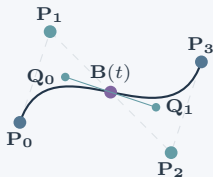
Step 1: Build two quadratic Béziers

$$Q_0(t) = \text{quadBezier}(P_0, P_1, P_2, t)$$

$$Q_1(t) = \text{quadBezier}(P_1, P_2, P_3, t)$$

Step 2: Interpolate between them

$$\begin{aligned} B(t) &= \text{lerp}(Q_0(t), Q_1(t), t) \\ &= (1-t)^3 P_0 + 3t(1-t)^2 P_1 \\ &\quad + 3t^2(1-t) P_2 + t^3 P_3 \end{aligned}$$



[[Desmos Demo: Cubic Bézier as LERP of Quadratics](#)]

General Bézier Curves

General Bézier Definition

General Formula

A Bézier curve of degree n with control points $\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_n$:

$$\mathbf{B}(t) = \sum_{i=0}^n B_{i,n}(t) \mathbf{P}_i$$

Where $B_{i,n}(t)$ are the Bernstein basis polynomials:

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

Degree vs Number of Control Points

- **Linear:** 2 control points, degree 1
- **Quadratic:** 3 control points, degree 2
- **Cubic:** 4 control points, degree 3
- **General:** $n + 1$ control points, degree n

Fundamental Properties

Bézier Properties

1. Endpoint Interpolation:

- $B(0) = P_0$
- $B(1) = P_n$

2. Convex Hull Property:

- Curve lies within convex hull of control points
- Provides predictable bounds

3. Transformation Invariance:

- Transform control points = transform curve
- Works with any coordinate system

Convex Hull



Why Bézier Curves Work So Well

Intuitive Control

Visual Predictability:

- Control points have geometric meaning
- Pulling a control point "attracts" the curve
- Natural, intuitive manipulation

Tangent Control:

- First and last segments of control polygon define tangents
- $B'(0) \propto (P_1 - P_0)$
- $B'(1) \propto (P_n - P_{n-1})$

Stability:

- Robust to numerical errors
- Consistent behavior



Bernstein Polynomials

Bernstein Basis Functions

Bernstein Polynomials

The i -th Bernstein polynomial of degree n :

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

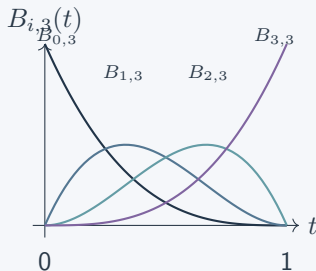
Examples for degree 3:

$$B_{0,3}(t) = (1-t)^3$$

$$B_{1,3}(t) = 3t(1-t)^2$$

$$B_{2,3}(t) = 3t^2(1-t)$$

$$B_{3,3}(t) = t^3$$



Properties

Key Properties

1. Non-negative:

$$B_{i,n}(t) \geq 0 \text{ for } t \in [0, 1]$$

Justification: For $t \in [0, 1]$ both t^i and $(1 - t)^{n-i}$ are non-negative. The binomial coefficient $\binom{n}{i}$ is also non-negative.

2. Partition of unity:

$$\sum_{i=0}^n B_{i,n}(t) = 1$$

Justification:

$$\begin{aligned} \sum_{i=0}^n B_{i,n}(t) &= \sum_{i=0}^n \binom{n}{i} t^i (1 - t)^{n-i} \\ &= (t + (1 - t))^n = 1^n = 1 \end{aligned}$$

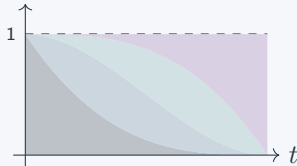
The equality follows directly from the binomial theorem applied with $x = t$ and $y = 1 - t$.

Non-negative



Always ≥ 0

Partition of Unity



Sum always equals 1

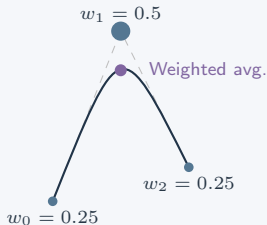
Bézier Curves as Convex Combinations

Geometric Interpretation

Convex Combination: Since Bernstein polynomials are non-negative and sum to 1:

$$\mathbf{B}(t) = \sum_{i=0}^n \underbrace{B_{i,n}(t)}_{\text{weights}} \mathbf{P}_i, \quad B_{i,n}(t) \geq 0, \quad \sum_{i=0}^n B_{i,n}(t) = 1$$

Any point on the curve is a *weighted average* of the control points.



The weights change smoothly with t , shifting the influence between control points.

Convex Hull Property

What is the Convex Hull?

The **convex hull** of a set of points is the smallest convex polygon containing them — imagine wrapping a rubber band around the points; the shape it makes is the convex hull.

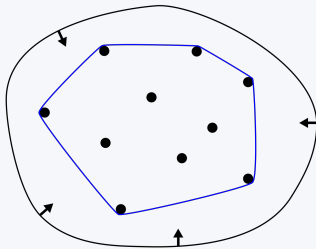


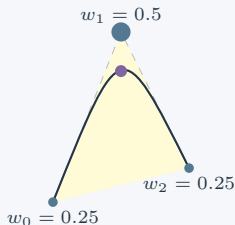
Figure 2: Convex hull of a set of points

Convex Hull Property

What is the Convex Hull?

The **convex hull** of a set of points is the smallest convex polygon containing them — imagine wrapping a rubber band around the points; the shape it makes is the convex hull.

Why it applies here: Bézier curve points are convex combinations of control points, so they must lie inside the convex hull of the control polygon.



This is exactly the **same idea as barycentric coordinates** in a triangle: any point with non-negative barycentric coordinates lies inside the triangle.

Why the Convex Hull Property Matters

Practical Consequences

1. Natural Bounds:

- Curve never leaves the convex hull

2. Stability:

- Small changes in control points → small changes in curve

3. Intuitive Control:

- Smooth, predictable shaping



Matrix Representation

Matrix Form for Efficiency

Cubic Bézier Matrix Form

For a cubic Bézier curve:

$$\mathbf{B}(t) = \begin{bmatrix} 1 & t & t^2 & t^3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{P}_0 \\ \mathbf{P}_1 \\ \mathbf{P}_2 \\ \mathbf{P}_3 \end{bmatrix}$$

Or more compactly:

$$\mathbf{B}(t) = \mathbf{T}(t) \cdot \mathbf{M}_B \cdot \mathbf{P}$$

Matrix Components

$\mathbf{T}(t)$: Parameter vector

$$\mathbf{T}(t) = \begin{bmatrix} 1 & t & t^2 & t^3 \end{bmatrix}$$

$\mathbf{M}_B$: Bézier basis matrix

$\mathbf{P}$: Control point vector

$$\mathbf{P} = \begin{bmatrix} \mathbf{P}_0 \\ \mathbf{P}_1 \\ \mathbf{P}_2 \\ \mathbf{P}_3 \end{bmatrix}$$

Why Use Matrix Form?

Computational Advantages

1. Efficiency:

- Single matrix multiplication
- GPU-friendly

2. Derivatives:

- Easy to compute tangents
- First derivative:

$$\mathbf{B}'(t) = \mathbf{T}'(t) \cdot \mathbf{M}_B \cdot \mathbf{P}$$

- Where

$$\mathbf{T}'(t) = \begin{bmatrix} 0 & 1 & 2t & 3t^2 \end{bmatrix}$$

3. Transformations:

- Apply transforms to control points
- Curve transforms automatically

Implementation Example

```
import numpy as np

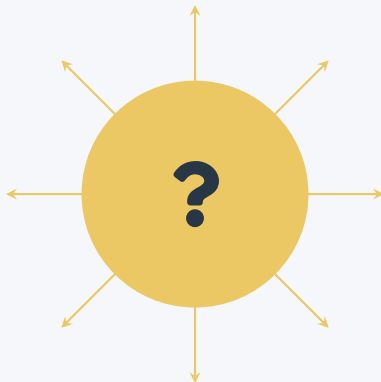
def bezier_cubic(t, control_points):
    # Parameter vector
    T = np.array([1, t, t**2, t**3])

    # Bézier basis matrix
    M = np.array([
        [ 1,  0,  0,  0],
        [-3,  3,  0,  0],
        [ 3, -6,  3,  0],
        [-1,  3, -3,  1]
    ])

    # Control points (4x2 for 2D)
    P = np.array(control_points)

    return T @ M @ P
```

Questions?



References & Further Reading



Peter Shirley and Steve Marschner et al. *Fundamentals of Computer Graphics (4th Edition)*. CRC Press, 2016.

Available as PDF



Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation (4th Edition)*. Morgan Kaufmann, 2023.

Available online



Peter Shirley. *Ray Tracing in One Weekend*. Self-published, 2016–2020.

Project Website



MIT OpenCourseWare: 6.837 Computer Graphics.
ocw.mit.edu/6-837



Scratchapixel: Learn Computer Graphics Programming.
scratchapixel.com